

포트폴리오

조성원

010-9742-4028

sungone000604@gmail.com

조성원

Game Client Programmer

+82 010-9742-4028

sungone000604@gmail.com

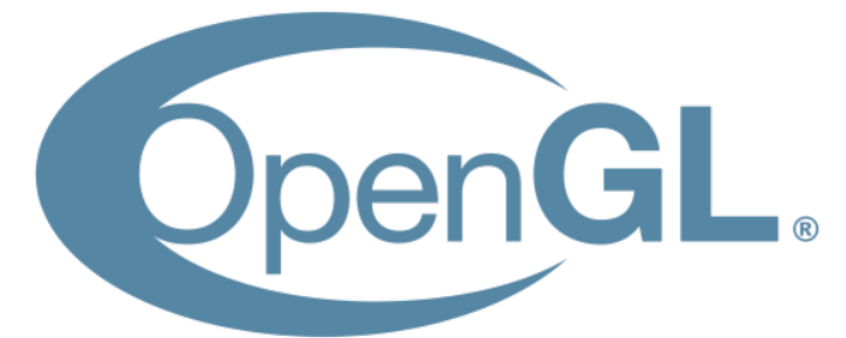
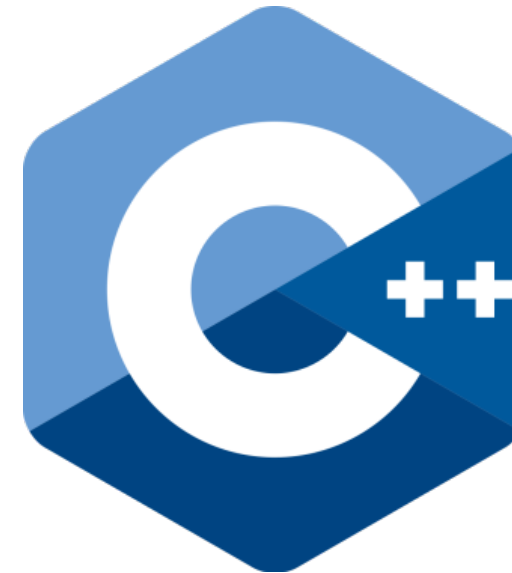


Education

2016.03 ~ 2019.02 - 광주진흥고등학교 졸업

2019.03 ~ 현재 - 한국공학대학교 게임공학과 재학중

Tech Stack



git



CONTENTS

01. 티유월드

4p : 게임 소개 , 5p : 구현 설명

02. Unity Of Mind

13p : 게임 소개 , 14p : 구현 설명

Project 소개

티유월드

2024.09 ~ 2025.07

한국공학대학교 졸업작품

- 3D MMORPG
- Unreal Engine 5.4 개발
- 팀 3명 : 클라이언트 1, 서버 1, 모델러 1
- 클라이언트 파트 담당
- 학생의 실수로 소환된 몬스터들을 물리치고 학교의 평화를 되찾는 액션 RPG 게임



링크



티유월드

플레이어 이동 동기화

내가 조종하는 플레이어 (Myplayer)
가 이동하면, 다른 모든 클라이언트에서 변경된
Myplayer 의 위치를 동기화 해야 합니다.

Myplayer 의 위치를 Server 로 보내기 위해서
네트워크 동기화 모듈 (NetworkSyncHandler) 을
만들어 변경된 위치/회전 정보를 서버로 전달하도록
구현하였습니다.

Myplayer 클래스 내부에서 패킷 전송 로직을 처리하지
않고 따로 전용 모듈을 이용함으로써 코드의 가독성이
더 좋아지고, 코드를 수정하거나 확장하는데 더 용이하
도록 하였습니다.

Myplayer.h

```
UPROPERTY()  
0 Blueprints에서 변경됨  
class UGPMYplayerNetworkSyncHandler* NetworkSyncHandler;
```

Myplayer.cpp : BeginPlay()

```
NetworkSyncHandler = NewObject<UGPMYplayerNetworkSyncHandler>(this, UGPMYplayerNetworkSyncHandler::StaticClass());  
if (NetworkSyncHandler)  
    NetworkSyncHandler->Initialize(this);
```

```
if (NetworkSyncHandler)  
    NetworkSyncHandler->Tick(DeltaTime);
```

Myplayer.cpp : Tick()

NetworkSync 모듈에서 일정 주기마다 위치/회전 패킷 전송 처리

```
void UGPMYplayerNetworkSyncHandler::Tick(float DeltaTime)  
{  
    if (!Owner || !Owner->NetMgr) return;  
  
    MovePacketSendTimer -= DeltaTime;  
  
    Owner->CharacterInfo.SetLocation(Owner->GetActorLocation().X, Owner->GetActorLocation().Y, Owner->GetActorLocation().Z);  
    Owner->CharacterInfo.Yaw = Owner->GetActorRotation().Yaw;  
  
    const float CurrentSpeed = Owner->GetVelocity().Size();  
    Owner->CharacterInfo.Stats.Speed = CurrentSpeed;  
  
    HandleIdleState();  
    Owner->UpdateGroundZLocation();  
    HandleJumpState();  
    HandleRotationChange();  
    HandlePeriodicSend();  
}
```

티유월드

플레이어 이동 동기화

너무 많은 위치/회전 패킷을 보내 서버에서 부하가 생기지 않게 하기 위해서 어떤 상황,조건에서 패킷을 보내야 하는지 고민을 하였습니다.

1. 플레이어가 움직이지 않는 상태에는 패킷을 보내 위치를 업데이트 할 필요가 없기 때문에 IDLE 상태인지 아닌지 확인하는 HandleIdleState() 를 호출합니다.
2. 플레이어가 위치해 있는 지면의 Z 값을 계산합니다.
3. 플레이어가 점프할 때 위치 패킷을 전송합니다.
 - 3-1. 점프가 끝나고 플레이어를 공중에서 지면으로 내리기 위해 공중 체류 보정 처리를 HandleJumpState() 에서 처리했습니다.
4. 플레이어가 제자리 회전할 때 회전 패킷을 전송합니다.
5. 플레이어가 IDLE 상태가 아니면 주기적으로 (0.5 ~ 1 초) 패킷을 전송합니다.

NetworkSyncHandler 변수

```
FVector LastLocation; // 마지막으로 확인한 캐릭터의 위치 벡터
float LastRotationYaw; // 마지막으로 확인한 캐릭터의 Yaw 회전값
FInfoData LastSendPlayerInfo; // 마지막으로 서버에 전송한 FInfoData 구조체
float MovePacketSendTimer = PAKCETSENDTIME; // 패킷 주기
bool bWasJumping = false; // 점프 중인지 확인
bool isJumpStart = false; // 점프를 시작했는지 확인
```

Tick() 에서 함수 호출 순서

```
// 1. MyPlayer 의 이동 여부(IDLE 상태) 판단
HandleIdleState();
// 2. MyPlayer 가 밟고 있는 지면의 값을 계산
Owner->UpdateGroundZLocation();
// 3-1. isJumpStart == true 일 때 서버로 위치패킷 전송
// 3-2. 공중체류 강제 보정 처리
HandleJumpState();
// 4. MyPlayer 이 가만히 회전할 때 서버로 회전패킷 전송
HandleRotationChange();
// 5. MyPlayer 가 움직이는 상태라면 (IDLE 이 아니면) 서버로 패킷 전송
HandlePeriodicSend();
```

티유월드

플레이어 이동 동기화

서버에서 받은 클라이언트들의 위치 정보는 0.5 ~ 1초 정도의 주기로 업데이트 되는데 이 주기마다 플레이어의 위치, 회전 값을 단순히 SetActorLocationAndRotation() 한다면 굉장히 부자연스럽게 보일 것입니다. 그래서 보간(Interpolation) 처리를 통해 자연스럽게 Player 를 움직이도록 구현했습니다.

현재 위치와 최종 위치 설정

Sprint 상태를 확인해서 속도 설정

목표 위치와 현재 위치의 거리를 계산

거리 차이가 거의 없으면 Return

방향만 필요하기 때문에 단위벡터로 설정

Speed X DeltaTime 만큼의 거리를 이동하는데 목표한 거리보다 멀리 가는것을 방지하기 위해 Min 함수 사용

회전 값도 RInterpTo() 를 통해 보간

보간 처리한 위치/회전 값을 Set

실제 이동량을 계산하여 속도 계산

```
void AGPCharacterBase::HandleRemoteMovementSync(float DeltaTime)
{
    if (DeltaTime <= 0.f)
        return;

    FVector CurrentLocation = GetActorLocation();
    FVector TargetLocation = CharacterInfo.Pos;

    const float SprintSpeed = 1200.f;
    const float WalkSpeed = 600.f;
    float Speed = CharacterInfo.HasState(STATE_RUN) ? SprintSpeed : WalkSpeed;

    FVector MoveDir = TargetLocation - CurrentLocation;
    float DistanceToTarget = MoveDir.Size();

    if (DistanceToTarget < KINDA_SMALL_NUMBER)
    {
        GetCharacterMovement()->Velocity = FVector::ZeroVector;
        return;
    }

    MoveDir.Normalize();

    float MoveDistance = FMath::Min(Speed * DeltaTime, DistanceToTarget);
    FVector NextLocation = CurrentLocation + MoveDir * MoveDistance;

    FRotator TargetRotation(0.f, CharacterInfo.Yaw, 0.f);
    FRotator InterpolatedRotation = FMath::RInterpTo(
        GetActorRotation(),
        TargetRotation,
        DeltaTime,
        10.f
    );

    SetActorLocationAndRotation(NextLocation, InterpolatedRotation, /*bSweep=*/ false);

    FVector NewLocation = GetActorLocation();
    FVector DeltaMove = (NewLocation - CurrentLocation);

    if (DeltaMove.SizeSquared() > KINDA_SMALL_NUMBER)
    {
        FVector NewVelocity = DeltaMove / DeltaTime;
        GetCharacterMovement()->Velocity = NewVelocity;
    }
    else
    {
        GetCharacterMovement()->Velocity = FVector::ZeroVector;
    }
}
```




전체 코드

Project 설명

티유월드

서버에서 수신받은 패킷들을 게임 월드에 반영

서버에서 수신한 여러 패킷들을 UGPObjectManager 클래스를 통해 게임 오브젝트에 반영하였습니다.

게임 월드 내의 모든 플레이어, 몬스터, 아이템은 UGPObjectManager 에서 고유 ID 와 객체 포인터로 매핑되어 관리

```
AGPCharacterMyplayer* MyPlayer;
TMap<int32, TWeakObjectPtr<AGPCharacterPlayer>> Players;
TMap<int32, TWeakObjectPtr<AGPCharacterMonster>> Monsters;
TMap<int32, TWeakObjectPtr<AGPItem>> Items;
TMap<uint32, FString> FriendMap;
TSet<uint32> RequestedFriendSet;
AGPCharacterMonster* Tino = nullptr;
```

플레이어 관련 패킷 처리 함수들

```
void SetMyPlayer(AGPCharacterMyplayer* InMyPlayer);
void AddMyPlayer(const FInfoData& PlayerInfo);
void AddPlayer(const FInfoData& PlayerInfo);
void RemovePlayer(int32 PlayerID);
void UpdatePlayer(const FInfoData& PlayerInfo);
void DamagedPlayer(const FInfoData& PlayerInfo);
void HandlePlayerDeath(int32 playerId);
void PlayerAttack(int32 PlayerId, FVector PlayerPos, float PlayerYaw);
void PlayerUseSkillStart(int32 PlayerID, ESkillGroup SkillGID, float PlayerYaw, FVector PlayerPos);
void PlayerUseSkillEnd(int32 PlayerID);
void SkillUnlock(ESkillGroup SkillGID);
void SkillUpgrade(ESkillGroup SkillGID);
void LevelUp(const FInfoData& PlayerInfo);
void RespawnMyPlayer(const FInfoData& info);
```

몬스터 관련 패킷 처리 함수들

```
void AddMonster(const FInfoData& MonsterInfo);
void RemoveMonster(int32 MonsterID);
void HandleMonsterDeath(int32 MonsterID);
void UpdateMonster(const FInfoData& MonsterInfo);
void DamagedMonster(const FInfoData& MonsterInfo, float Damage);
bool IsCritical(float Damage);
void PlayEarthQuakeEffect(const FVector& RockPos, bool bDebug);
void PlayFlameBreathEffect(const FVector& Origin, const FVector& Dir, float Range, float Angle, bool bDebug);
```

아이템/인벤토리 관련 패킷 처리 함수들

```
void ItemSpawn(uint32 ItemID, uint8 ItemType, FVector Pos);
void ItemPickUp(uint32 ItemID);
void ItemDespawn(uint32 ItemID);
void DropItem(uint32 ItemID, uint8 ItemType, FVector Pos);
void AddInventoryItem(uint32 ItemID, uint8 ItemType);
void UseInventoryItem(uint32 ItemID);
void EquipItem(int32 PlayerID, uint8 ItemType);
void UnequipItem(int32 PlayerID, uint8 ItemType);
void SoundWhenBuy(ResultCode Code);
```

채널/레벨 이동 관련 패킷 처리 함수들

```
void ChangeChannel(const FVector& RandomPos);
void ChangeZone(ZoneType OldZone, ZoneType NewZone, const FVector& RandomPos);
bool IsChangingZone() const { return bChangingZone; }
void SetChangingZone(bool value) { bChangingZone = value; }
FRotator GetDefaultZoneRotation(ZoneType Zone);
UFUNCTION()
0개의 청사진 참조
void HandleLevelUnloaded();
UFUNCTION()
void HandleLevelLoaded();
```

퀘스트 관련 패킷 처리 함수들

```
void ShowTutorialStartQuest();
void OnQuestStart(QuestType Quest);
void OnQuestReward(QuestType Quest, bool bSuccess, uint32 ExpReward, uint32 GoldReward);
void PlayWorldIntro();
void PlayTinoIntro();
UFUNCTION()
void OnWorldIntroFinished();
UFUNCTION()
void OnTinoIntroFinished();
```

친구 시스템 패킷 처리 함수들

```
void AddRequestFriend(const FFriendInfo& FriendInfo);
void RemoveRequestFriend(uint32 DBId);
void AddFriend(const FFriendInfo& FriendInfo);
void RemoveFriend(uint32 DBId);
void ShowFriendMessage(int32 Result);
uint32 GetFriendDBId(FString Name) const;
```

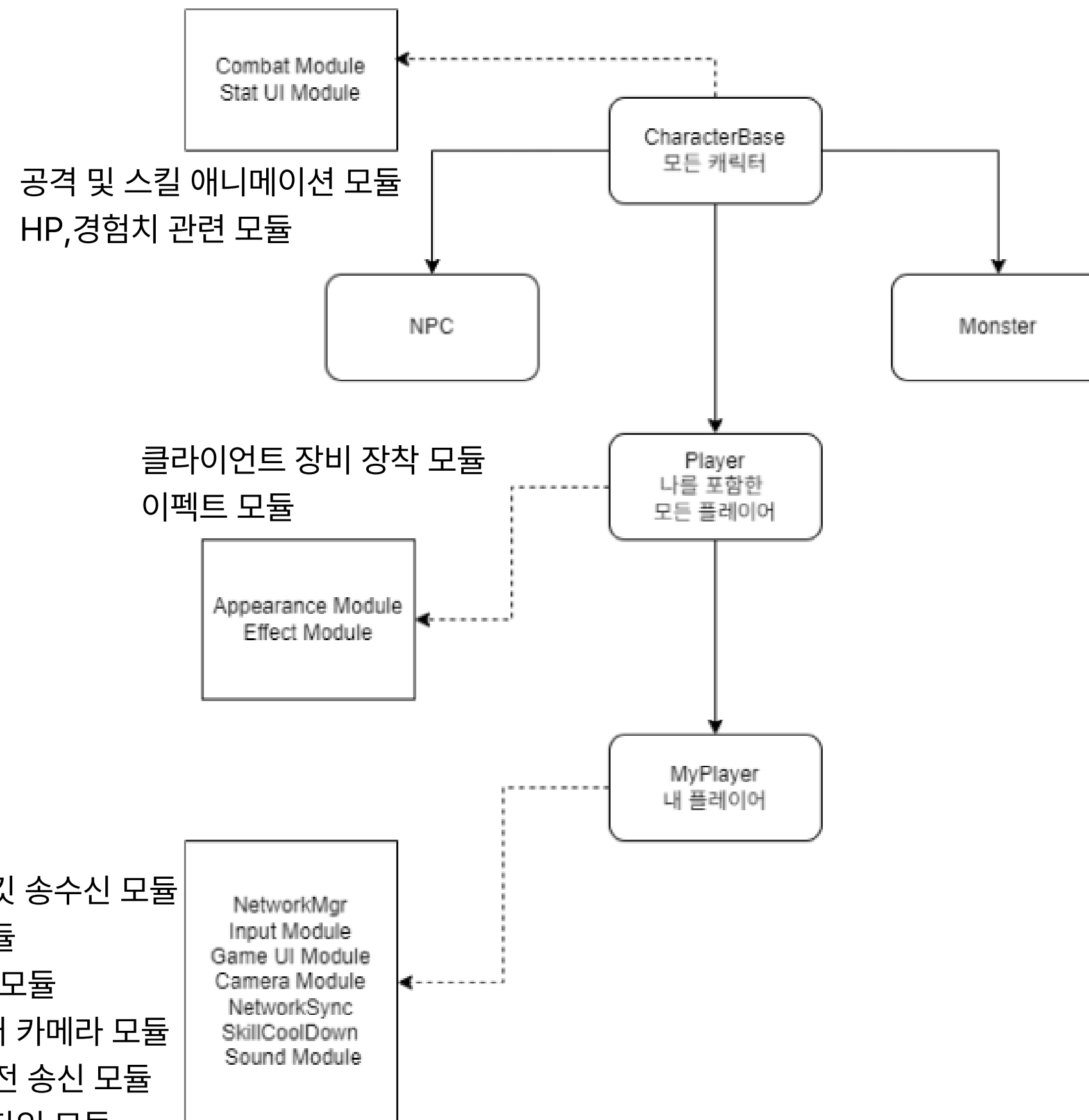

티유월드

캐릭터 클래스 구조 설계 및 캐릭터 기능 모듈화

MMORPG 의 다양한 캐릭터가 공통적으로 가지는 기능은 AGPCharacterBase 에서 정의하고, 이를 상속 받아 플레이어, 몬스터, NPC 로 세분화하였습니다. 내 플레이어와 다른 클라이언트들을 구분하기 위해 AGPCharacterMyplayer 클래스를 별도로 설계하였습니다.

캐릭터 클래스 내부에 모든 기능을 직접 구현하지 않고 전투, 스탯, UI 등 기능 단위를 모듈화하여 각 캐릭터 클래스가 필요한 모듈만 조합하도록 설계하였습니다. 이를 통해 캐릭터별 특성을 잘 반영할 수 있도록 하였습니다.

서버 패킷 송수신 모듈
입력 모듈
여러 UI 모듈
플레이어 카메라 모듈
이동/회전 송신 모듈
스킬 쿨타임 모듈
사운드 모듈





티유월드

캐릭터 전투 시스템 구현

UGPCharacterCombatHandler 모듈을 통해 플레이어/몬스터의 전투 시스템을 구현했습니다. 이 모듈은 기본 공격, 스킬, 피격, 사망 처리와 관련된 애니메이션을 몽타주 단위로 관리하며, 공격 속도 증가, 돌진형 스킬, 멀티 Hit 스킬의 특성에 따른 전투 기능을 타이머와 상태 플래그로 제어해서 구현했습니다. 또한 플레이어의 공격 상태를 판단해서 서버, 클라이언트간 동기화가 원활히 이루어지도록 설계하였습니다.

AnimMontage 변수

```
UPROPERTY()
0 Blueprints에서 변경됨
UAnimMontage* AttackWithoutWeaponMontage;
UPROPERTY()
0 Blueprints에서 변경됨
UAnimMontage* AttackMontage;
UPROPERTY()
0 Blueprints에서 변경됨
UAnimMontage* WarriorAttackMontage_1;
UPROPERTY()
0 Blueprints에서 변경됨
UAnimMontage* WarriorAttackMontage_2;
UPROPERTY()
0 Blueprints에서 변경됨
UAnimMontage* QSkillMontage;
UPROPERTY()
0 Blueprints에서 변경됨
UAnimMontage* ESkillMontage;
UPROPERTY()
0 Blueprints에서 변경됨
UAnimMontage* RSkillMontage;
UPROPERTY()
0 Blueprints에서 변경됨
UAnimMontage* DeadMontage;
UPROPERTY()
0 Blueprints에서 변경됨
UAnimMontage* MonsterHitMontage;
```

몽타주 재생

```
// 기본공격 애니메이션
void PlayAutoAttackMontage();
void OnAutoAttackMontageEnded(UAnimMontage* Montage, bool bInterrupted);
// 사망 애니메이션
void HandleDeath();
void PlayDeadAnimation();
void SetDeadMontage(UAnimMontage* Montage);
void SetDeadEventDelay(float Delay);
// 몬스터가 맞았을 때 애니메이션
void PlayMonsterHitMontage();
// 스킬 애니메이션
void PlayQSkillMontage();
void PlayESkillMontage();
void PlayRSkillMontage();
void OnSkillMontageEnded(UAnimMontage* Montage, bool bInterrupted);
```

플레이어 스킬 어빌리티

```
// 거너 캐릭터 R스킬 어빌리티
void ApplyAttackSpeedBoost(float BoostPlayRate, float Duration);
void ResetAttackSpeed();
float DefaultPlayRate = 2.0f;
FTimerHandle AttackSpeedResetTimer;
// 워리어 캐릭터 E스킬 어빌리티
void StartDash();
void FinishDash();
void UpdateDash();
FTimerHandle DashTimerHandle;
bool bIsDashing = false;
float DashDistance = 600.f;
float DashDuration = 1.f;
float DashElapsedTime = 0.0f;
FVector DashStartLocation;
FVector DashEndLocation;
// 워리어 캐릭터 R스킬 어빌리티
void PlayMultiHitSkill(int32 HitCount, float Interval);
void ExecuteMultiHit();
int32 RemainingHits = 0;
float HitInterval = 0.5f;
FTimerHandle MultiHitTimerHandle;
```

Project 설명

티유월드

인벤토리 및 상점 시스템 구현

Item 객체에 필요한 속성들을 모아
Struct 로 만들고 데이터 테이블을 만들
어 아이템을 관리하였습니다.

몬스터 처치 시 스폰되는 아이템입니다.

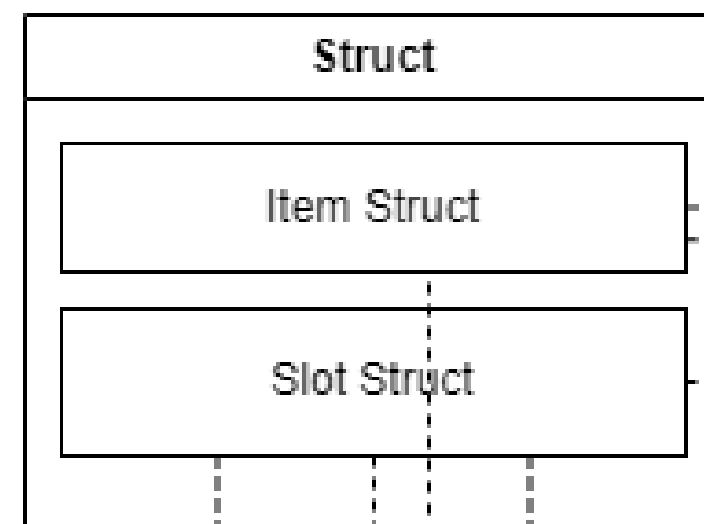


인벤토리 창, 상점 창에서 슬롯 하나를 담
당하는 클래스입니다.



ItemSlot 클래스를 인벤토리와 상
점 양쪽에서 공통적으로 사용하도록
구조화하였습니다.

Slot 을 구성하는 데 필요한 속성들을 모아
Struct 로 만들어 관리하였습니다.



player get item

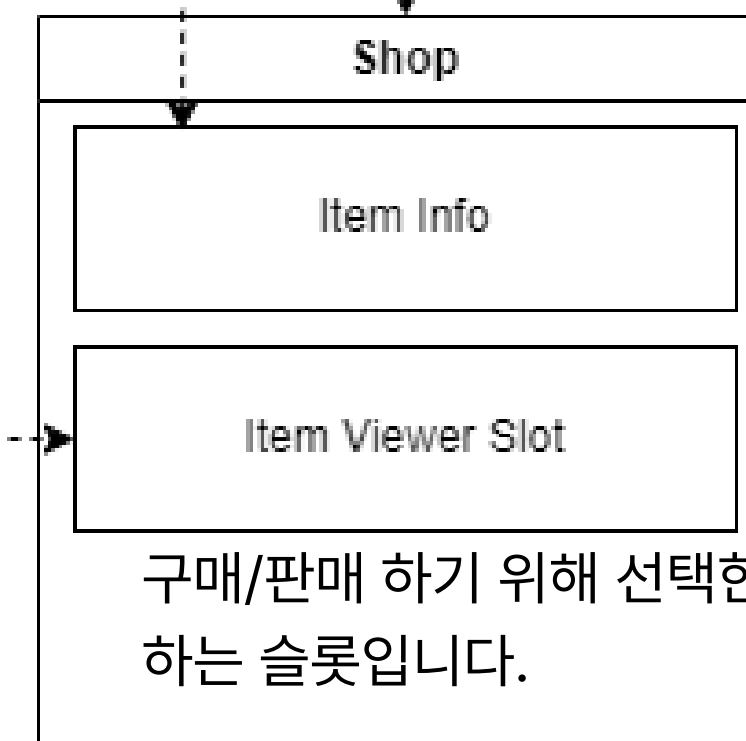
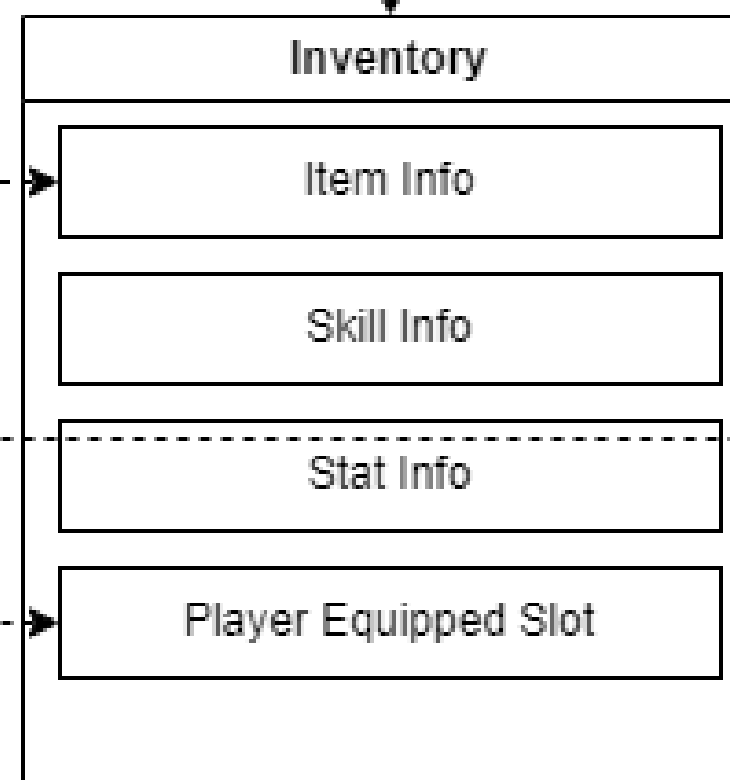
shop display item

Item Info : 아이템 슬롯 위에 마우스 커서
를 올리면 아이템 정보를 표시합니다.

Skill Info : 스킬 이름, 레벨, 데미지 계수 등
의 정보를 인벤토리에 표시하였습니다.

Skill Info : 플레이어의 현재 스탯 정보를
인벤토리에 표시하였습니다.

Player Equipped Slot : 플레이어가 현재
장착한 무기를 슬롯에 등록해서 한눈에 알
수 있도록 구현하였습니다.

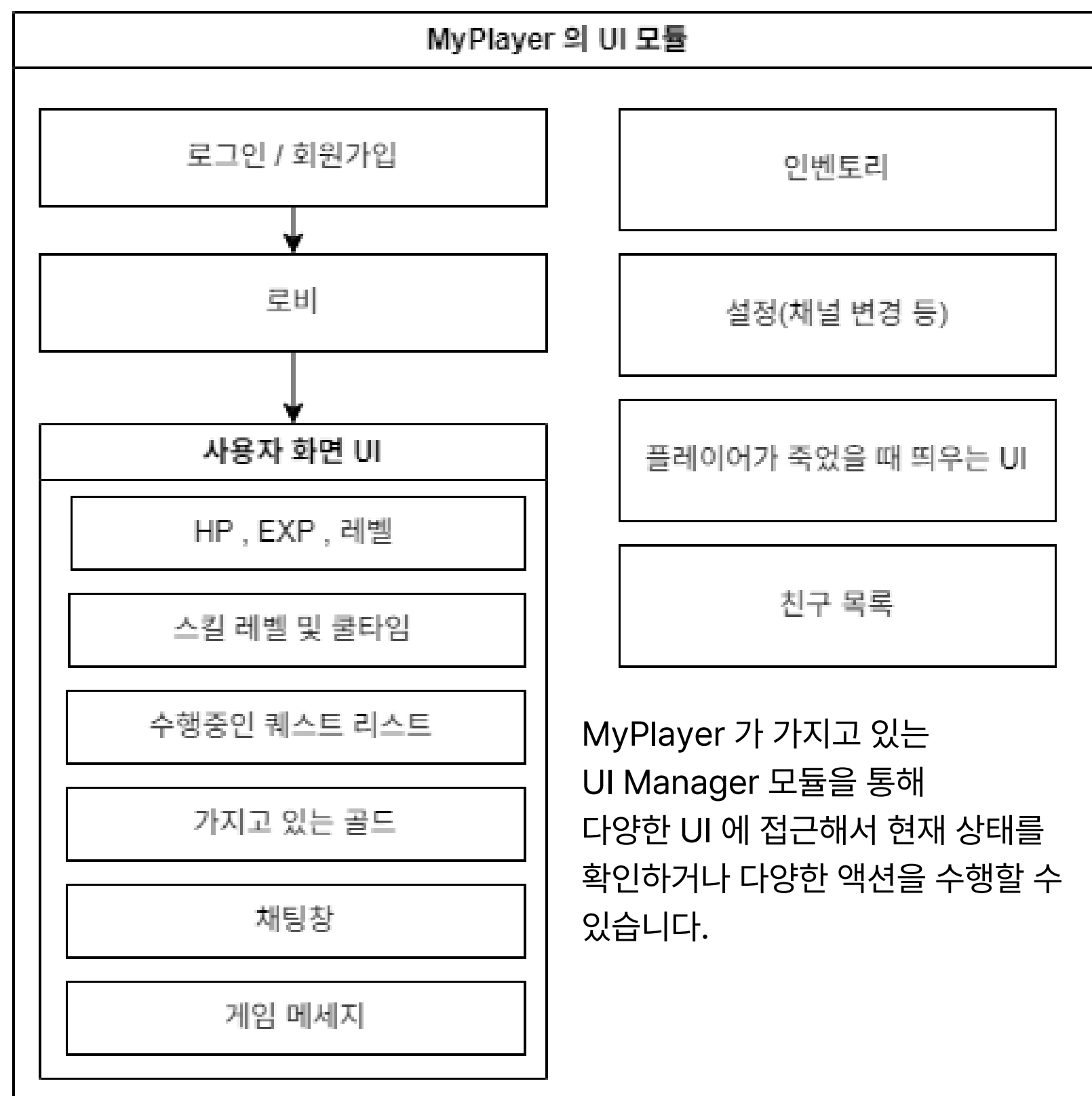


구매/판매 하기 위해 선택한 아이템을 표시
하는 슬롯입니다.

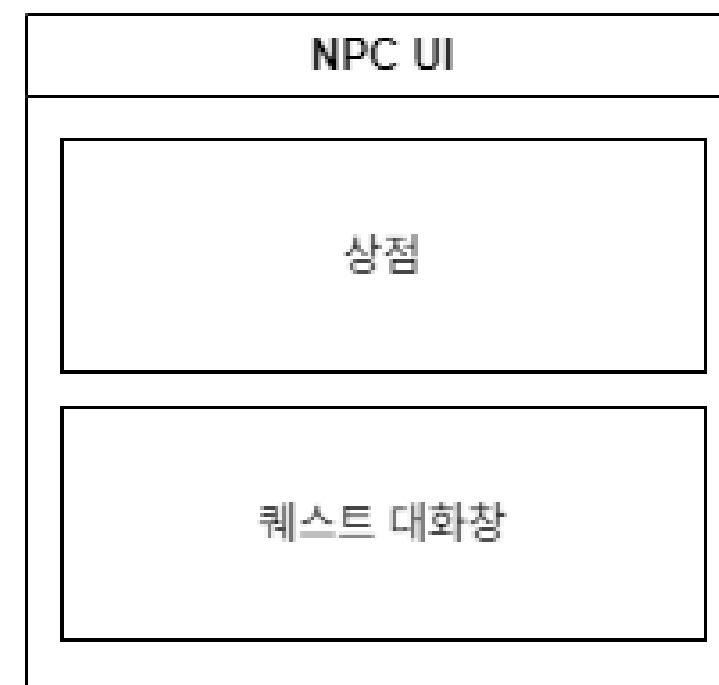
Project 설명

티유월드

UI 구현



NPC 와 상호작용하여 UI 를 열고닫을 수 있습니다.



Project 소개

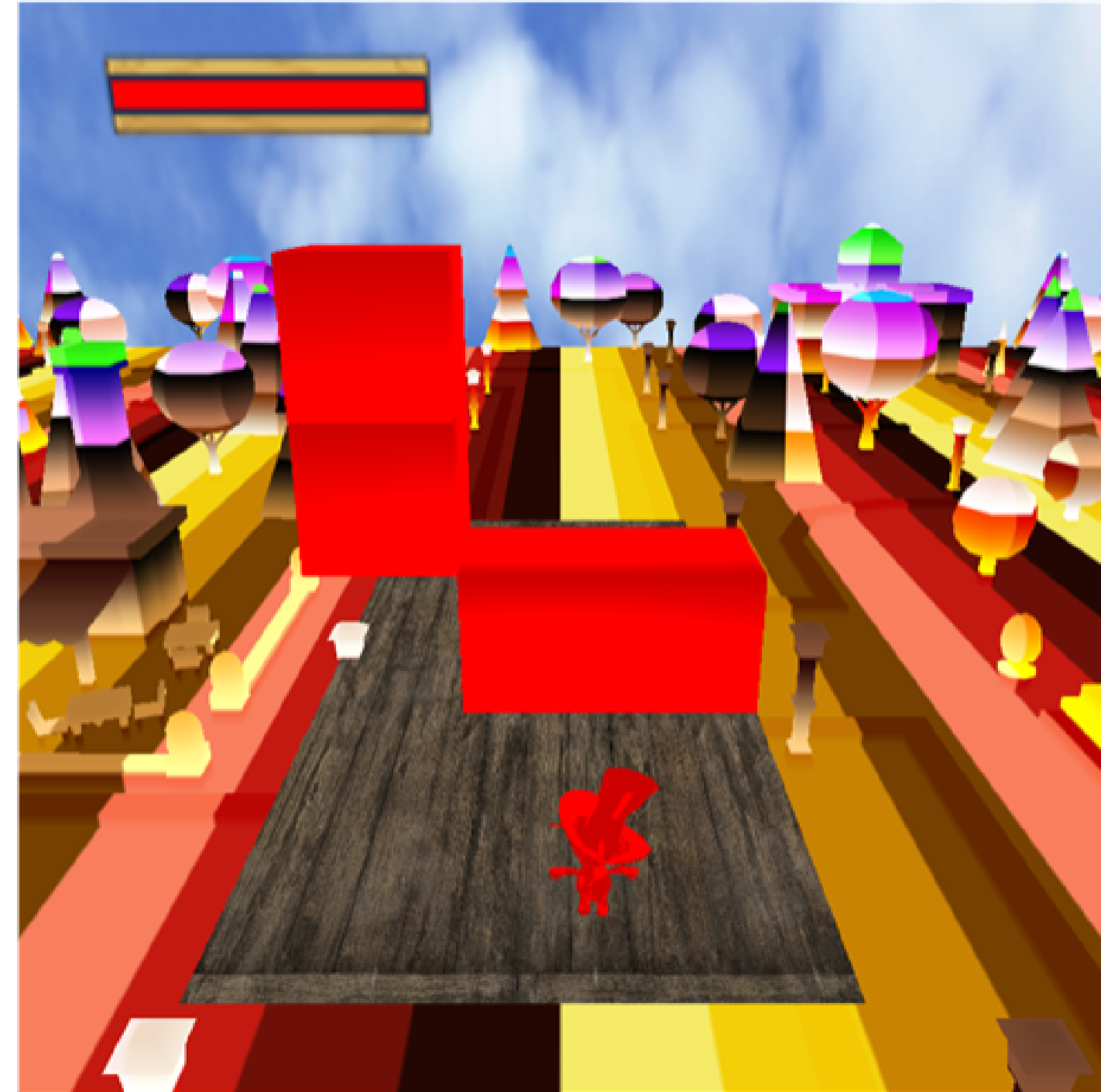
Unity Of Mind

2023.09 ~ 2023.12

컴퓨터그래픽스 텀프로젝트

- 벽 피하기 게임
- OpenGL 개발
- 팀 2명 : 클라이언트 2
- 오브젝트 , 셰이더 , 게임 로직 담당
- 다가오는 벽을 피해서 끝까지 생존하는 게임

링크

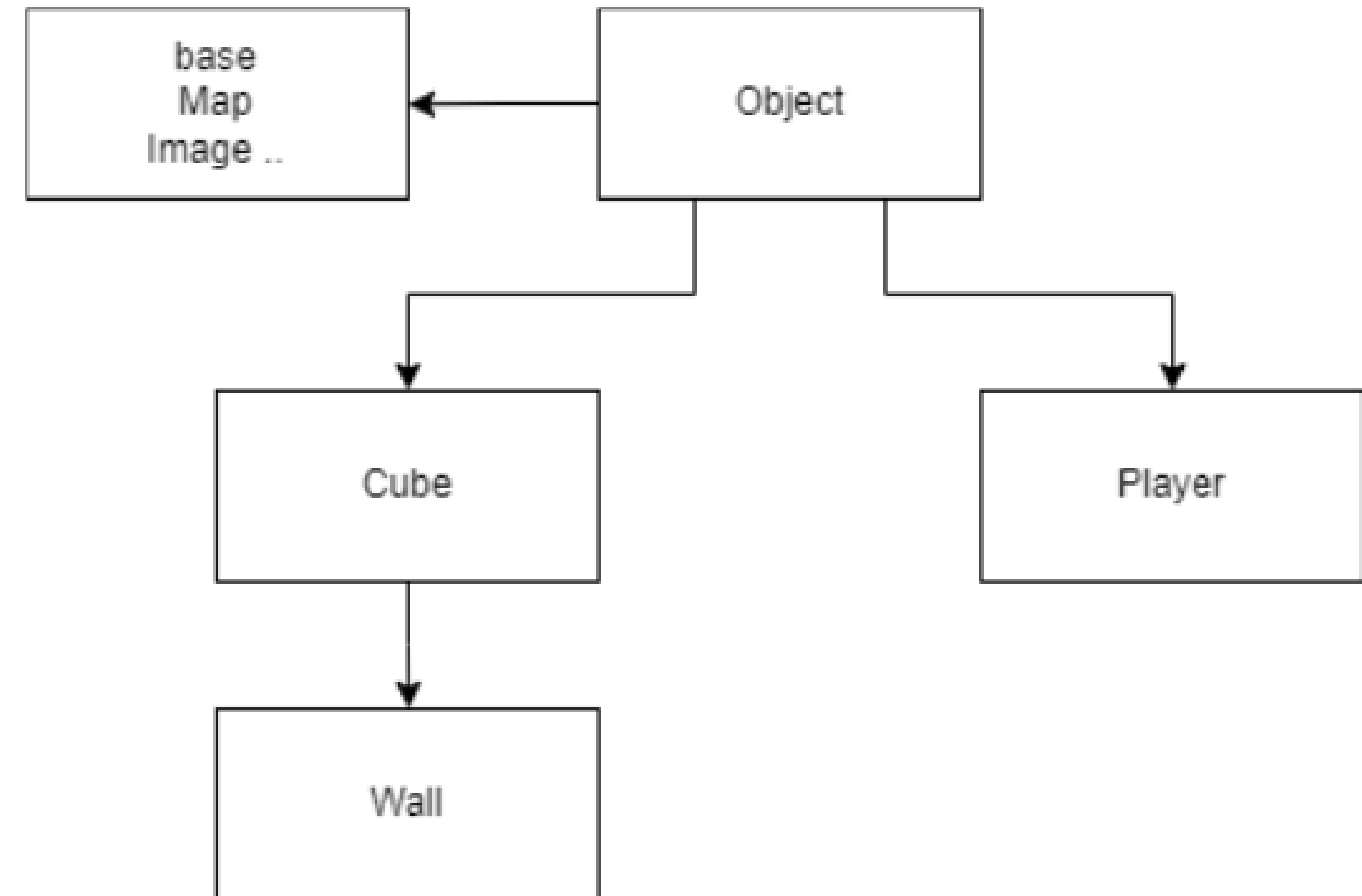


Unity Of Mind

게임 오브젝트 설계

게임의 핵심 오브젝트들을 Object 기반 클래스에서 상속받아 Cube, Wall, Player 등으로 구현하였습니다.

공통 속성과 기능은 Object에서 관리하고, 각 파생 클래스는 개별 동작을 오버라이드하도록 설계하였습니다.



오브젝트 코드

https://github.com/sungone/Unity_Of_Mind/blob/main/CG_FinalProject/object.h

Cube, Wall 코드

https://github.com/sungone/Unity_Of_Mind/blob/main/CG_FinalProject/wall.h

Player 코드

https://github.com/sungone/Unity_Of_Mind/blob/main/CG_FinalProject/player.h

Unity Of Mind

쉐이더 관리 모듈 설계

버텍스 및 프래그먼트 셰이더를 작성하여 오브젝트의 색상과 텍스처를 처리할 수 있도록 했습니다.
이를 통해 Cube, Wall, Player 등 다양한 오브젝트를 렌더링하는 셰이더 시스템을 구축하였습니다.

쉐이더 코드

https://github.com/sungone/Unity_Of_Mind/blob/main/CG_FinalProject/shader.cpp

```
void makeVertexShaders(char* file)
{
    GLchar* vertexSource;

    // char fileName[] = "vertex.glsl";
    // char *file = fileName;
    vertexSource = fileToBuf(file);

    vertexShader = glCreateShader(GL_VERTEX_SHADER);
    glShaderSource(vertexShader, 1, &vertexSource, NULL);
    glCompileShader(vertexShader);

    GLint result;
    GLchar errorLog[512];
    glGetShaderiv(vertexShader, GL_COMPILE_STATUS, &result);
    if (!result)
    {
        glGetShaderInfoLog(vertexShader, 512, NULL, errorLog);
        std::cerr << "Error compiling vertex shader: " << errorLog << std::endl;
        return;
    }
}

void makeFragmentShaders(char* file)
{
    GLchar* fragmentSource;

    // char fileName[] = "fragment.glsl";
    // char *file = fileName;
    fragmentSource = fileToBuf(file);

    fragmentShader = glCreateShader(GL_FRAGMENT_SHADER);
    glShaderSource(fragmentShader, 1, &fragmentSource, NULL);
    glCompileShader(fragmentShader);

    GLint result;
    GLchar errorLog[512];
    glGetShaderiv(fragmentShader, GL_COMPILE_STATUS, &result);
    if (!result)
    {
        glGetShaderInfoLog(fragmentShader, 512, NULL, errorLog);
        std::cerr << "Error compiling fragment shader: " << errorLog << std::endl;
        return;
    }
}
```

감사합니다

조성원

010-9742-4028

sungone000604@gmail.com